KAUNAS UNIVERSITY OF TECHNOLOGY

FACULTY OF INFORMATICS

DEPARTMENT OF COMPUTER SCIENCES

The First Principles of Digital Logic (P175B100 English Students)

Report of laboratory works

Lab work No. 08

Author of the report:
Student of IFDU-5 Group
Riyash Elangovan

Teacher:
dr. Plauska Ignas

KAUNAS, 2026

TABLE OF CONTENTS

1. **Personal assignments for lab work……. 3**

2. **Data conversion……. 4**

3. **Create a Karnaugh map and find grouped data…… 6**

4. **Convert to NAND symbol…...……. 9**

5. **Draw a Schematics……..... 9**

6. **Check Simulation Wizard…… 12**

1. Personal assignments for lab work

The Boolean function is defined by a set of decimal minterms for six input variables a, b, c, d, e, f.

$$F(a, b, c, d, e, f) = \sum m(0,2,6,15,24,32,33,34,35,36,37,38,39,42,49,57,60,61)$$

Variable order is abcdef (a=MSB, f=LSB).

These minterms represent input combinations for which the output of the function is equal to logic 1.

2. Data conversion

Since the function contains six input variables, there are $2^6 = 64$ possible input combinations. Each decimal minterm is converted into a 6-bit binary representation corresponding to the values of variables a through f

	a	b	c	d	e	f
0	0	0	0	0	0	0
2	0	0	0	0	1	0
6	0	0	0	1	1	0
15	0	0	1	1	1	1
24	0	1	1	0	0	0
32	1	0	0	0	0	0
33	1	0	0	0	0	1
34	1	0	0	0	1	0
35	1	0	0	0	1	1
36	1	0	0	1	0	0
37	1	0	0	1	0	1
38	1	0	0	1	1	0
39	1	0	0	1	1	1
42	1	0	1	0	1	0
49	1	1	0	0	0	1
57	1	1	1	0	0	1
60	1	1	1	1	0	0
61	1	1	1	1	0	1

TRUTH TABLE

A truth table is constructed where the output is set to 1 for the specified minterms and 0 for all other combinations.

3. Create a Karnaugh map and find grouped data

A Karnaugh map is used to simplify the Boolean function by grouping adjacent cells containing logic 1. The cells are arranged using Gray code ordering so that neighboring cells differ in only one variable.

Because the function has six variables, multiple four-variable Karnaugh maps are used to represent all possible combinations.

Cells corresponding to the given minterms are marked with logic 1, and adjacent cells are grouped in powers of two to eliminate changing variables and obtain simplified product terms.

abc \ def	000	001	011	010	110	111	101	100
000	1			1	1			
001						1		
011	1							
010								
110		1						
111		1					1	1
101				1				
100	1	1	1	1	1	1	1	1

Group 1	000	001	011	010	110	111	101	100
000	1			1	1			
001						1		
011	1							
010								
110		1						
111		1					1	1
101				1				
100	1	1	1	1	1	1	1	1

Group 2	000	001	011	010	110	111	101	100
000	1			1	1			
001						1		
011	1							
010								
110		1						
111		1					1	1
101				1				
100	1	1	1	1	1	1	1	1

Group 3								
	000	001	011	010	110	111	101	100
000	1			1	1			
001						1		
011	1							
010								
110		1						
111		1					1	1
101				1				
100	1	1	1	1	1	1	1	1

Group 4								
	000	001	011	010	110	111	101	100
000	1			1	1			
001						1		
011	1							
010								
110		1						
111		1					1	1
101				1				
100	1	1	1	1	1	1	1	1

Group 5								
	000	001	011	010	110	111	101	100
000	1			1	1			
001						1		
011	1							
010								
110		1						
111		1					1	1
101				1				
100	1	1	1	1	1	1	1	1

Group 6								
	000	001	011	010	110	111	101	100
000	1			1	1			
001						1		
011	1							
010								
110		1						
111		1					1	1
101				1				
100	1	1	1	1	1	1	1	1

Group 7								
	000	001	011	010	110	111	101	100
000	1			1	1			
001						1		
011	1							
010								
110		1						
111		1					1	1
101					1			
100	1	1	1	1	1	1	1	1

Group 8								
	000	001	011	010	110	111	101	100
000	1			1	1			
001						1		
011	1							
010								
110		1						
111		1					1	1
101					1			
100	1	1	1	1	1	1	1	1

Simplified Boolean Expression:-

After grouping the cells, the minimized Boolean expression is obtained as the sum of the simplified product terms derived from each group.

$$F = \bar{b}\bar{c}\bar{f}(\bar{a}e + \bar{d}\bar{e}) + bc\bar{e}(\bar{a}\bar{d}\bar{f} + ad) + f(\bar{a}\bar{b}c\,de + ab\bar{d}\bar{e}) + a\bar{b}\bar{c} + a\bar{b}\bar{d}\bar{f}$$

4. Conversion to NAND Implementation:-

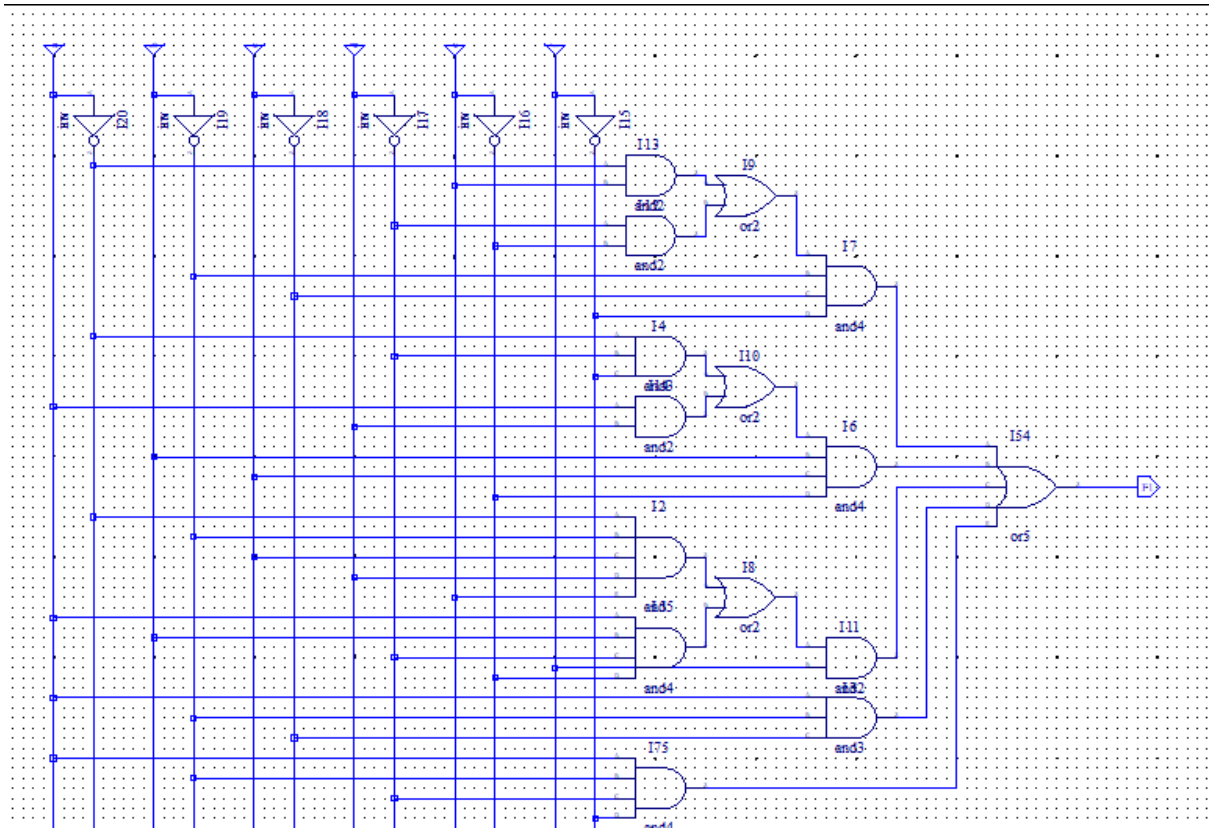
The NAND gate is a universal logic gate, which means that any Boolean function can be implemented using only NAND gates.
Using De Morgan's theorem:

$$X + Y = (X' \cdot Y')'$$

the minimized expression is transformed into a NAND-only form by replacing each logical operation with its NAND equivalent.

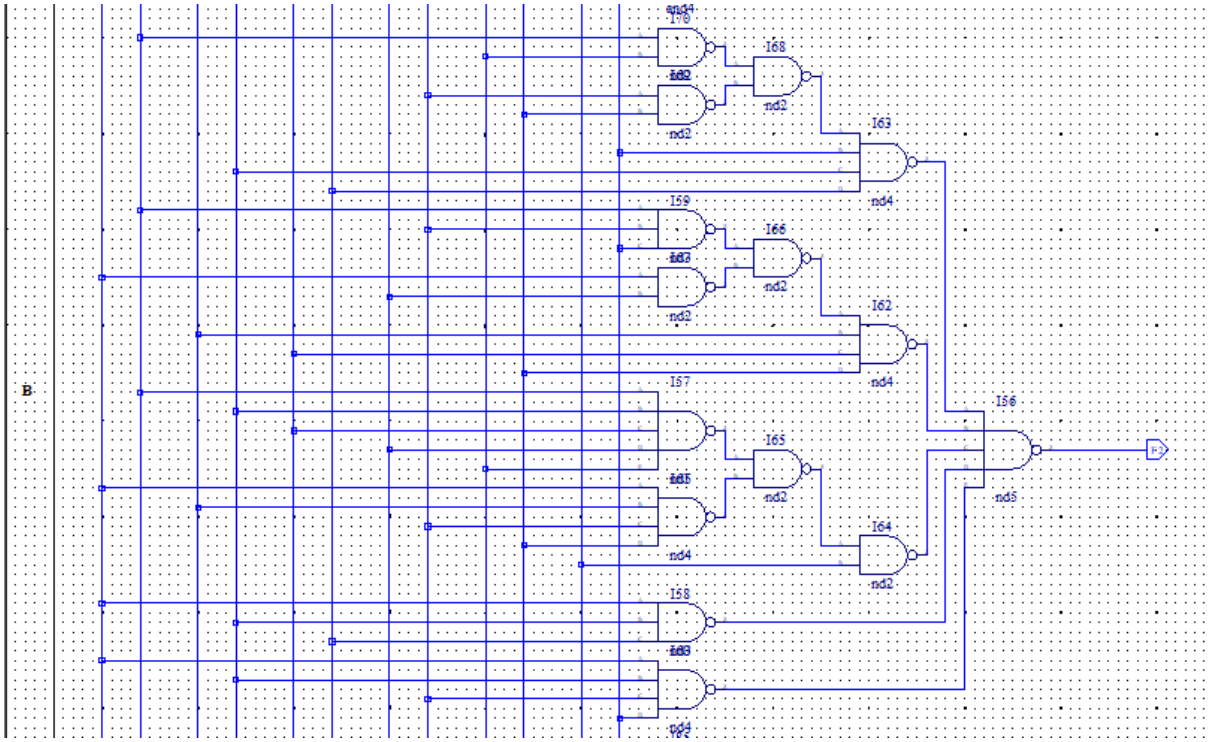
5. Schematic Design

A digital schematic implementing the NAND-only expression was constructed using logic design software. The input variables a, b, c, d, e, f were connected to the circuit, and the output corresponds to the minimized Boolean function.



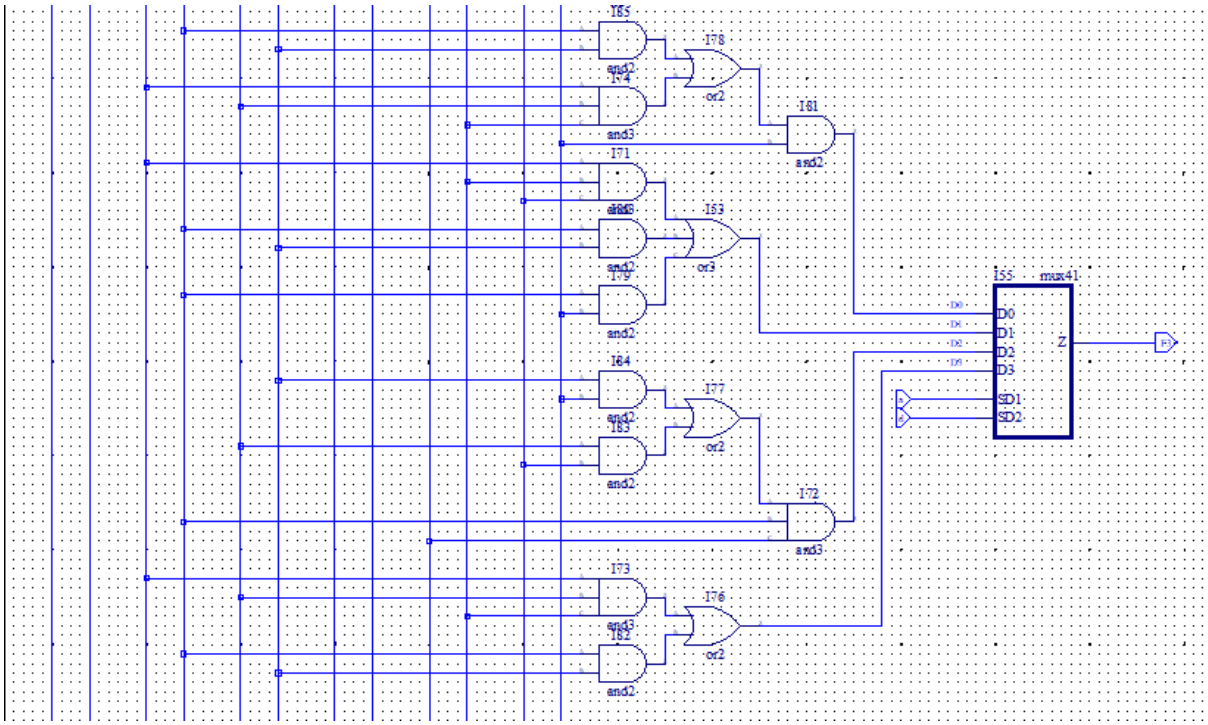
Schematic 1

The first circuit was built using AND–OR–NOT gates directly from the minimized expression.



Schematic 2

The second circuit implements the same function using NAND gates only, applying De Morgan's laws and NAND-inverters.



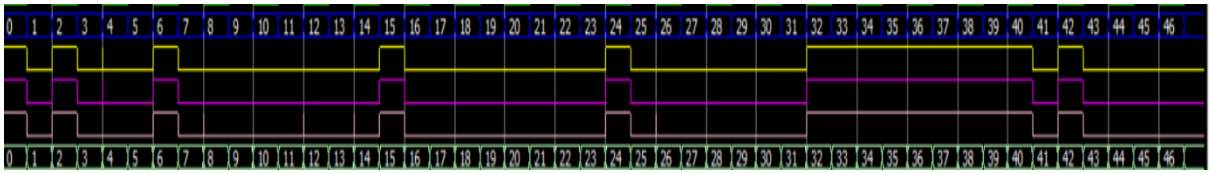
Schematic 3

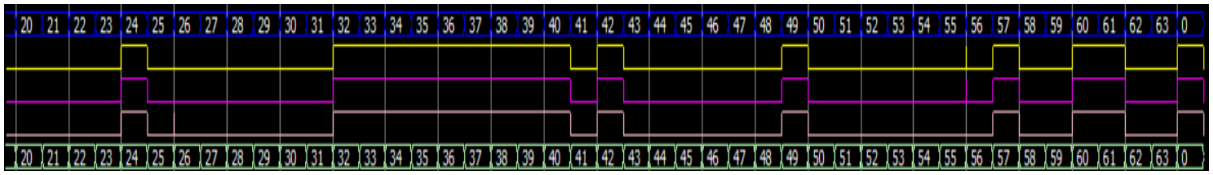
The third circuit uses a 4:1 multiplexer, where two variables are used as select signals and the data inputs D_0–D_3 are formed from the remaining variables.

Figures 1–3 show the three implementations.

6. Simulation and Verification

Simulation was performed by applying all $2^6 = 64$input combinations in the variable order $abcdef$. The output F was checked against the required minterms $0,2,6,15,24,32,33,34,35,36,37,38,39,42,49,57,60,61$. The waveform confirms that all three implementations produce identical outputs and the function is equal to 1 only for the specified minterms.





Conclusion

In this laboratory work, a six-variable Boolean function defined by a set of minterms was successfully analyzed and minimized using Karnaugh map techniques. The simplified expression was implemented using NAND gates, and the resulting circuit was verified through simulation. The results confirm that the designed circuit operates correctly for all input combinations.